# 森林火灾研究完整代码与复现证据

森林火灾后植被绿度与树冠覆盖的变化差异及数学预测

本文件列出指标计算、二次函数、幅度上限校正、评分、作图与文件生成的完整程序，并提供实际数据摘录。全部可执行UTF-8源码和原始表随资料包提供。

## 一 复现入口

保持analysis/forest_recovery与output/forest_recovery_report的相对目录。运行顺序及依赖见README.md；指标和单位见data_dictionary.md。数值复算使用保存的实际观测，不需要登录GEE；重新采集卫星资料需要使用自己的账户和固定位置文件。

输入包含103,764条位置年度记录、9,564个固定位置、81场火灾及636个火灾年度。36场用于拟合和按起火年份交叉验证，37场用于回顾性检验，8场用于补充核对。第一年作为校正输入，误差只评价第2—8年。

所有数据图证均为实际CSV记录的可视化摘录，不是GEE网页截图，也不是新增的地面观测。产品重提取核查表为保存的实际记录，运行本包不表示重新访问卫星平台。

## 二 指标和正式函数

LG为DIRECT参照校准的绿色覆盖估计。ZP_TCC=TCC/100，为树冠覆盖代理。两者的受灾／对照年度比值分别除以灾前三年基准；DE=R_ZP_TCC − R_LG。物理ZP可以通过地面逐树调查测量，但不参与时间预测。

LG二次函数[a,b,c]：[0.457185239473671, 0.1004725844743174, -0.0065836926051989755]

ZP_TCC二次函数[a,b,c]：[0.5498805537484331, 0.016052485771525005, -0.0012342007658844864]

首年偏差e_k=R_k(1) − f_k(1)，校正后Rhat_k(t)=f_k(t)+s_k(t)。ZP首年在[0.46,0.64]外，s_ZP=e_ZP；区间内s_ZP=0.25e_ZP。只有区间外且abs(e_LG)≥0.12时，s_LG=sign(e_LG)×min(abs(e_LG),abs(s_ZP))；其他情形s_LG=e_LG×0.5^(t − 1)。DE预测由两项相减。

37场回顾性检验RMSE：二次函数LG12.661、ZP13.651、DE9.592点；幅度上限校正8.561、8.421、8.596点。训练按起火年份交叉验证RMSE分别为10.668、11.290、8.710点，以及9.261、7.082、8.877点。训练DE没有改善；结果不能视为普遍预测优势。

校正规则的确定使用了研究资料，检验属于回顾性分析。经验半宽只用训练交叉验证绝对误差计算，检验资料用于报告实际覆盖率。首年误差不计入改善，真实缺测不替换为预测。

# 三 固定位置记录

WILLOW固定受灾位置的卫星记录

| year | t | ndvi | evi | ndvi_n | tcc30 | tcc30_mapped_fraction |
|---|---|---|---|---|---|---|
| 2001.0000 | -3.0000 | 0.5251 | 0.2662 | 6 | 35.4655 | 1.0000 |
| 2002.0000 | -2.0000 | 0.4229 | 0.2211 | 6 | 34.3477 | 1.0000 |
| 2003.0000 | -1.0000 | 0.4786 | 0.2378 | 6 | 33.6984 | 1.0000 |
| 2005.0000 | 1.0000 | 0.3906 | 0.1923 | 6 | 21.7543 | 1.0000 |
| 2006.0000 | 2.0000 | 0.4093 | 0.2141 | 6 | 24.5754 | 1.0000 |
| 2012.0000 | 8.0000 | 0.4088 | 0.2246 | 6 | 26.5883 | 1.0000 |

实际CSV摘录；位置ID：AZ3410911149820040624_impact_42115_26794。TCC为百分比。

源：data/satellite_pixel_years.csv。位置ID、质量次数和覆盖率均可逐行核对。

# 四 组均值与年度状态

典型火灾的组均值与相对状态

| 火灾 | 火后年 | 受灾LG | 对照LG | 受灾TCC | 对照TCC | LG状态 | ZP状态 | DE |
|---|---|---|---|---|---|---|---|---|
| DAY | 1 | 0.1244 | 0.4503 | 12.5542 | 52.5180 | 26.5174 | 23.0511 | -3.4662 |
| DAY | 2 | 0.0855 | 0.4535 | 10.2394 | 52.9429 | 18.0989 | 18.6500 | 0.5511 |
| DAY | 8 | 0.2891 | 0.4407 | 15.7377 | 51.5445 | 62.9736 | 29.4422 | -33.5314 |
| WILLOW | 1 | 0.1748 | 0.4151 | 16.9968 | 45.7058 | 39.3029 | 37.4417 | -1.8613 |
| WILLOW | 2 | 0.2464 | 0.3863 | 21.0291 | 45.6397 | 59.5261 | 46.3913 | -13.1348 |
| WILLOW | 8 | 0.2895 | 0.4084 | 28.4629 | 44.5947 | 66.1461 | 64.2621 | -1.8841 |
| WHISKEY COMPLEX | 1 | 0.4006 | 0.5217 | 39.7497 | 54.8467 | 72.9961 | 70.4859 | -2.5102 |
| WHISKEY COMPLEX | 2 | 0.4592 | 0.5218 | 41.1453 | 54.3358 | 83.6565 | 73.6467 | -10.0098 |
| WHISKEY COMPLEX | 8 | 0.5502 | 0.5381 | 42.1055 | 54.6988 | 97.2031 | 74.8652 | -22.3379 |

LG原始值为比例，TCC为百分比，状态与DE为点。完整记录保存在CSV中。

源：data/group_year_values.csv及rebuilt_annual_states.csv。先算受灾／对照比值，再除以三年基准。

# 五 灾前基准

灾前三年基准核对

| event_name | baseline_LG | baseline_TCC | baseline_CV_LG | baseline_CV_TCC |
|---|---|---|---|---|
| WILLOW | 1.0714 | 0.9932 | 0.0253 | 0.0200 |
| WHISKEY COMPLEX | 1.0519 | 1.0282 | 0.0120 | 0.0091 |
| DAY | 1.0417 | 1.0370 | 0.0149 | 0.0137 |

CV为三年样本标准差／均值绝对值。

源：data/baseline_values.csv。不同指标分别使用自己的基准，不共用分母。

# 六 近期火灾记录

PARK的观测与真实缺测

| group | year | t | LG | ZP_TCC | n_fixed | n_LG | n_TCC |
|---|---|---|---|---|---|---|---|
| control | 2021 | -3 | 0.5354 | 0.6145 | 60 | 60 | 60 |
| impact | 2021 | -3 | 0.5302 | 0.5474 | 60 | 60 | 60 |
| control | 2022 | -2 | 0.5169 | 0.6244 | 60 | 60 | 60 |
| impact | 2022 | -2 | 0.5254 | 0.5687 | 60 | 60 | 60 |
| control | 2023 | -1 | 0.5181 | 0.6282 | 60 | 60 | 60 |
| impact | 2023 | -1 | 0.5306 | 0.5735 | 60 | 60 | 60 |
| control | 2025 | 1 | 0.5161 | 0.6111 | 60 | 60 | 60 |
| impact | 2025 | 1 | 0.1447 | 0.1750 | 60 | 60 | 60 |
| control | 2026 | 2 | 0.5349 | 缺测 | 60 | 60 | 0 |
| impact | 2026 | 2 | 0.1943 | 缺测 | 60 | 60 | 0 |

2026年LG已有观测，同一产品版本的TCC缺测。观测空值未用预测填充。

源：data/recent_group_values.csv。与数据库37场检验分别核对。

# 七 相同样本评分

37场回顾性检验的评分输出

| method | target | n_fires | n_states | RMSE | observed_SD | R2 |
|---|---|---|---|---|---|---|
| calibrated | DE | 37 | 259 | 8.5959 | 10.0048 | 0.2618 |
| calibrated | LG | 37 | 259 | 8.5615 | 13.8179 | 0.6161 |
| calibrated | ZP_TCC | 37 | 259 | 8.4206 | 13.6833 | 0.6213 |
| quadratic | DE | 37 | 259 | 9.5921 | 10.0048 | 0.0808 |
| quadratic | LG | 37 | 259 | 12.6613 | 13.8179 | 0.1604 |
| quadratic | ZP_TCC | 37 | 259 | 13.6509 | 13.6833 | 0.0047 |

quadratic为二次函数；calibrated为幅度上限校正。第2—8年，每目标259个状态。

源：results/model_metrics.csv。RMSE与SD采用相同火灾等权；$R^2 = 1 - RMSE^2/SD^2$。

# 八 81场火灾的用途

| 火灾 | 起火年 | 用途 |
|---|---|---|
| HAM LAKE | 2007 | 补充观测核对 |
| ARABIA BAY | 2011 | 补充观测核对 |
| EAST TEXAS COMPLEX | 2011 | 补充观测核对 |
| MOUNT PLEASANT | 2016 | 补充观测核对 |
| CARR | 2018 | 补充观测核对 |
| CREEK | 2020 | 补充观测核对 |
| CALDOR | 2021 | 补充观测核对 |
| DIXIE | 2021 | 补充观测核对 |
| NUTALL COMPLEX (NUTALL) | 2004 | 回顾性检验 |
| WILLOW | 2004 | 回顾性检验 |
| BLOSSOM | 2005 | 回顾性检验 |
| SIGNAL ROCK | 2005 | 回顾性检验 |
| COLUMBIA COMPLEX | 2006 | 回顾性检验 |
| JUNGLE | 2006 | 回顾性检验 |
| RED EAGLE | 2006 | 回顾性检验 |
| TATOOSH COMPLEX | 2006 | 回顾性检验 |
| BRUSH CREEK | 2007 | 回顾性检验 |
| CASTLE ROCK | 2007 | 回顾性检验 |
| CHIPPY CREEK | 2007 | 回顾性检验 |
| CONFLUENCE COMPLEX (CLEAR SAGE) | 2007 | 回顾性检验 |
| ELK COMPLEX (KING CREEK 2) | 2007 | 回顾性检验 |
| GRAYS CREEK | 2007 | 回顾性检验 |
| JOCKO LAKES | 2007 | 回顾性检验 |
| MANILA CREEK | 2007 | 回顾性检验 |
| CHALK | 2008 | 回顾性检验 |
| CORRAL | 2008 | 回顾性检验 |
| IRON AND ALPS COMPLEXES (IRONSIDE) | 2008 | 回顾性检验 |
| RATTLE | 2008 | 回顾性检验 |
| YOLLA BOLLY COMPLEX (IRON) | 2008 | 回顾性检验 |
| BRUSHY MOUNTAIN | 2009 | 回顾性检验 |
| HIGHWAY 31 | 2009 | 回顾性检验 |
| LA BREA | 2009 | 回顾性检验 |
| NORTH FORK COMPLEX | 2009 | 回顾性检验 |
| SCHULTZ | 2010 | 回顾性检验 |

| 火灾 | 起火年 | 用途 |
|---|---|---|
| CHIPS | 2012 | 回顾性检验 |
| NORTH PASS | 2012 | 回顾性检验 |
| DAY | 2014 | 回顾性检验 |
| HAPPY CAMP COMPLEX | 2014 | 回顾性检验 |
| WHISKEY COMPLEX | 2014 | 回顾性检验 |
| ELDORADO | 2015 | 回顾性检验 |
| TOWER | 2015 | 回顾性检验 |
| COPPER KING | 2016 | 回顾性检验 |
| BRIANHEAD | 2017 | 回顾性检验 |
| HANOVER | 2017 | 回顾性检验 |
| SCALP | 2017 | 回顾性检验 |
| POWER | 2004 | 函数拟合与交叉验证 |
| I-90 COMPLEX (TARKIO) | 2005 | 函数拟合与交叉验证 |
| VALLEY ROAD | 2005 | 函数拟合与交叉验证 |
| CAVITY LAKE | 2006 | 函数拟合与交叉验证 |
| HEAVENS GATE COMPLEX (BLACK BUTTE) | 2006 | 函数拟合与交叉验证 |
| SOUTHFORK COMPLEX (COUGAR) | 2006 | 函数拟合与交叉验证 |
| THORN CREEK | 2006 | 函数拟合与交叉验证 |
| CORPORAL | 2007 | 函数拟合与交叉验证 |
| MOONLIGHT | 2007 | 函数拟合与交叉验证 |
| RAILLEY MOUNTAIN | 2007 | 函数拟合与交叉验证 |
| ROMBO MOUNTAIN | 2007 | 函数拟合与交叉验证 |
| SHOWER BATH COMPLEX (SHOWER BATH) | 2007 | 函数拟合与交叉验证 |
| SLIDE | 2007 | 函数拟合与交叉验证 |
| CANYON COMPLEX (SCOTCH) | 2008 | 函数拟合与交叉验证 |
| IRON AND ALPS COMPLEXES (EAGLE) | 2008 | 函数拟合与交叉验证 |
| LIME COMPLEX (LIME) | 2008 | 函数拟合与交叉验证 |
| LIME COMPLEX (MINERS) | 2008 | 函数拟合与交叉验证 |
| SHU LIGHTNING COMPLEX (MOON) | 2008 | 函数拟合与交叉验证 |
| FREEZE OUT | 2012 | 函数拟合与交叉验证 |
| MILLIE | 2012 | 函数拟合与交叉验证 |
| MUSTANG COMPLEX | 2012 | 函数拟合与交叉验证 |
| READING | 2012 | 函数拟合与交叉验证 |
| WHITEWATER-BALDY | 2012 | 函数拟合与交叉验证 |
| AMERICAN | 2013 | 函数拟合与交叉验证 |
| MILE MARKER 28 | 2013 | 函数拟合与交叉验证 |

| 火灾 | 起火年 | 用途 |
|---|---|---|
| RIM | 2013 | 函数拟合与交叉验证 |
| WHISKEY | 2013 | 函数拟合与交叉验证 |
| KING | 2014 | 函数拟合与交叉验证 |
| MAN | 2014 | 函数拟合与交叉验证 |
| ROUGH | 2015 | 函数拟合与交叉验证 |
| SLIDE | 2015 | 函数拟合与交叉验证 |
| TRAIL CREEK | 2015 | 函数拟合与交叉验证 |
| WOLVERINE | 2015 | 函数拟合与交叉验证 |
| DEEP CREEK | 2017 | 函数拟合与交叉验证 |
| HELENA | 2017 | 函数拟合与交叉验证 |
| JULY | 2017 | 函数拟合与交叉验证 |

# 九 代码与数据校验

SHA-256核对的是文件字节内容，不证明生态测量无误。全包校验值见MANIFEST_SHA256.json。

analysis/forest_recovery/data/satellite_pixel_years.csv

505bfe8da313ba79aa4fb79d01b244f9ce8d9f6d6eba48d15579bf385c7d9d8c

analysis/forest_recovery/data/group_year_values.csv

ad5f6a14e3665c957c0ee139828067cb003637e8ca3bbc4509a5596751aa088d

analysis/forest_recovery/data/rebuilt_annual_states.csv

04a70e75b35d5c9863c60924a05fad9db4d33bad6a62cb014f44e393d2d41f71

analysis/forest_recovery/data/baseline_values.csv

a5632cb171173bf56813b09d0a1e2d1130b41d420523a629190c831875b3926f

analysis/forest_recovery/data/recent_group_values.csv

c0b8112238ae1976e38f9c558bbabbfd9c91a52053dc80121322f1dcafe23aa7

analysis/forest_recovery/results/model_metrics.csv

c4e3500ebcc28b2e5ec446cf78ad334b422405ab1ca03ace9e17a9b42a1d6e36

analysis/forest_recovery/data/measurement/product_reextraction_checks.csv

dac41492ec92768b08ddf5159769a5e9becad49bcd020bd39634d12a3ddc69d4

# 十 完整代码

下面所有程序未省略函数或代码行。PDF按阅读宽度自动折行；运行与复制应使用资料包同名源文件。固定参数见model.json；干净的独立报告版式见assets/report_layout.docx。

01 model.py

02 rebuild_measurements.py

03 check_green_calibration.py

04 analyze.py

05 predict.py

06 make_figures.py

07 build_report.py

08 build_manual.py

09 verify.py

10 collect_satellite.js

11 collect_green_reference.js

12 build_code_evidence.py

13 package_release.py

# 代码01 model.py

SHA-256 9115412e9363f41da743f4ba13e70f66c4cd285ff2de4deafd279bb8d14afc3a

```
"""Two prediction methods for post-fire relative LG and tree canopy cover."""
from pathlib import Path
import json
import numpy as np
import pandas as pd

R=Path(__file__).resolve().parent
CONFIG=json.loads((R/'model.json').read_text())
TARGETS={'LG':'R_LG','ZP_TCC':'R_ZP_TCC','DE':'DE'}
METHODS=['quadratic','calibrated']
COEF=np.array([CONFIG['coefficients'][k] for k in ['LG','ZP_TCC']])

def fit_quadratic(d):
    t=d.t.to_numpy(float);X=np.c_[np.ones(len(t)),t,t*t]
    w=1/d.groupby('event_id').t.transform('size').to_numpy(float)
    return np.linalg.lstsq(X*np.sqrt(w[:,None]),d[['R_LG','R_ZP_TCC']].to_numpy()*np.sqrt(w[:,None]),rcond=None)[0].T

def quadratic(coef,t):
    t=np.atleast_1d(np.asarray(t,float))
    return np.c_[np.ones(len(t)),t,t*t]@np.asarray(coef).T

def predict(first,t,method='calibrated',coef=COEF):
    t=np.atleast_1d(np.asarray(t,float));first=np.asarray(first,float)
    if method not in METHODS:raise ValueError('Choose quadratic or calibrated.')
    if np.any((t<1)|(t>8)):raise ValueError('Prediction window is fire years1-8.')
    p=quadratic(coef,t);e=first-quadratic(coef,[1])[0]
    lo,hi=CONFIG['ZP_first_year_interval'];outside=bool(first[1]<lo or first[1]>hi)
    cap=bool(outside and abs(e[0])>=CONFIG['LG_first_year_error_threshold'])
    if method=='calibrated':
        sZ=e[1]*(1 if outside else CONFIG['inside_ZP_shift_fraction'])
        sG=e[0]*CONFIG['LG_decay_factor']**(t-1)
        if cap:sG=np.full(len(t),np.sign(e[0])*min(abs(e[0]),abs(sZ)))
        p=p+np.c_[sG,np.repeat(sZ,len(t))]
    return np.c_[p,p[:,1]-p[:,0]],dict(ZP_outside=outside,LG_cap=cap)

def event_predictions(g,coef,method):
    r1=g[g.t.eq(1)].iloc[0];t=np.arange(1,9)
    p,flags=predict([r1.R_LG,r1.R_ZP_TCC],t,method,coef);obs=g.set_index('t');rows=[]
    for j,age in enumerate(t):
        for k,(target,col) in enumerate(TARGETS.items()):
            rows.append(dict(event_id=r1.event_id,event_name=r1.event_name,event_year=int(r1.event_year),
                t=int(age),calendar_year=int(r1.event_year+age),target=target,method=method,
                prediction=float(p[j,k]),observed=float(obs.loc[age,col]) if age in obs.index else np.nan,**flags))
    return pd.DataFrame(rows)

def metrics(g):
    g=g[g.t.ge(2)&np.isfinite(g.observed)&np.isfinite(g.prediction)]
    w=1/g.groupby('event_id').t.transform('size').to_numpy(float);w/=w.sum()
    y=g.observed.to_numpy();e=g.prediction.to_numpy()-y
    mse=float(w@(e*e));variance=float(w@((y-w@y)**2))
    return dict(n_fires=g.event_id.nunique(),n_states=len(g),RMSE=float(np.sqrt(mse)),
        MAE=float(w@abs(e)),bias=float(w@e),observed_SD=float(np.sqrt(variance)),
        R2=1-mse/variance if variance>0 else np.nan)
```

# 代码02 rebuild_measurements.py

SHA-256 c9eb5a7ba0aef26d11720a1b5604ef21678269b222fca608c746aefc5adaf890

```
"""Compute annual indicators from satellite pixel-year records and fixed sites."""
import hashlib,json
import numpy as np,pandas as pd
from model import R,CONFIG

def main():
    raw=pd.read_csv(R/'data/satellite_pixel_years.csv',low_memory=False)
    events=pd.read_csv(R/'data/events.csv').set_index('event_id')
    assert not raw.duplicated(['pixel_id','year']).any()
    assert (raw.year-raw.event_year).eq(raw.t).all()
    b=CONFIG['LG_measurement_coefficients']
    raw['LG']=np.clip(b[0]+b[1]*raw.ndvi+b[2]*raw.evi,0,1)
    raw['ZP_TCC']=raw.tcc30/100
    states=[];groups=[];baselines=[]
    for eid,a in raw.groupby('event_id',sort=False):
        info=events.loc[eid];n=a.drop_duplicates('pixel_id').groupby('group').size()
        means=a.groupby(['group','t'])[['LG','ZP_TCC']].mean()
        counts=a.groupby(['group','t'])[['LG','ZP_TCC']].count()
        for group in ['impact','control']:
            assert n[group]==info['n_fixed_'+group]
            means.loc[group]=means.loc[group].where(counts.loc[group].eq(n[group])).to_numpy()
        q=(means.loc['impact']/means.loc['control']).replace([np.inf,-np.inf],np.nan)
        base=q.loc[[-3,-2,-1]].mean(skipna=False);relative=q/base
        cv=q.loc[[-3,-2,-1]].std(ddof=1)/abs(base)
        for (group,t),v in means.iterrows():
            groups.append(dict(event_id=eid,event_name=info.event_name,event_year=int(info.event_y
ear),
                group=group,t=int(t),year=int(info.event_year+t),n_fixed=int(n[group]),
                n_LG=int(counts.loc[(group,t),'LG']),n_TCC=int(counts.loc[(group,t),'ZP_TCC']),
                LG=float(v.LG),ZP_TCC=float(v.ZP_TCC)))
        for t,v in relative[relative.index>0].iterrows():
            states.append(dict(event_id=eid,event_name=info.event_name,event_year=int(info.event_y
ear),
                t=int(t),year=int(info.event_year+t),R_LG=float(v.LG),R_ZP_TCC=float(v.ZP_TCC),DE=
float(v.ZP_TCC-v.LG)))
        baselines.append(dict(event_id=eid,event_name=info.event_name,baseline_LG=float(base.LG),
            baseline_TCC=float(base.ZP_TCC),baseline_CV_LG=float(cv.LG),baseline_CV_TCC=float(cv.Z
P_TCC)))
    pd.DataFrame(states).to_csv(R/'data/rebuilt_annual_states.csv',index=False)
    pd.DataFrame(groups).to_csv(R/'data/group_year_values.csv',index=False)
    pd.DataFrame(baselines).to_csv(R/'data/baseline_values.csv',index=False)
    result=dict(n_pixel_years=len(raw),n_fixed_positions=raw.pixel_id.nunique(),n_fires=len(events
),
        n_event_years=len(states),source_SHA256=hashlib.sha256((R/'data/satellite_pixel_years.csv'
).read_bytes()).hexdigest())
    (R/'results/measurement_counts.json').write_text(json.dumps(result,indent=2));print(result)

if __name__=='__main__':main()
```

# 代码03 check_green_calibration.py

SHA-256 d51d3c5b7095b6d4023e342c58ff3f756cf58471f9532bbac848b2cd6302a127

```
"""Reproduce the fixed LG measurement calibration; do not choose time models."""
import json
import numpy as np
import pandas as pd
from model import R,CONFIG

def main():
    d=pd.read_csv(R/'data/measurement/direct_calibration_subset.csv')
    X=d[['NDVI','EVI']].to_numpy();y=d.reference.to_numpy()
    w=1/d.groupby('site_group').site_group.transform('size').to_numpy(float)
    mu=np.average(X,axis=0,weights=w)
    sd=np.sqrt(np.average((X-mu)**2,axis=0,weights=w))
    A=np.c_[np.ones(len(d)),(X-mu)/sd]
    penalty=np.c_[np.zeros((2,1)),np.eye(2)]
    beta=np.linalg.lstsq(np.r_[A*np.sqrt(w[:,None]),penalty],
        np.r_[y*np.sqrt(w),np.zeros(2)],rcond=None)[0]
    slopes=beta[1:]/sd;coef=np.r_[beta[0]-mu@slopes,slopes]
    assert (slopes>=0).all()
    difference=float(np.max(abs(coef-CONFIG['LG_measurement_coefficients'])))
    assert difference<1e-12
    prediction=np.clip(coef[0]+X@slopes,0,1)
    errors=pd.DataFrame({'site_group':d.site_group,'se':(prediction-y)**2})
    result=dict(n_references=len(d),n_spatial_groups=d.site_group.nunique(),
        raw_unit_coefficients=coef.tolist(),standardization_mean=mu.tolist(),
        standardization_SD=sd.tolist(),standardized_slope_penalty=1,
        maximum_coefficient_difference=difference,
        calibration_fit_RMSE=float(np.sqrt(errors.groupby('site_group').se.mean().mean())),
        calibration_fit_is_independent_validation=False)
    (R/'results/green_calibration_check.json').write_text(json.dumps(result,indent=2))
    print(json.dumps(result,indent=2))

if __name__=='__main__':main()
```

# 代码04 analyze.py

SHA-256 23dfcf4de8d093a5f4df05c05ca9d840ff97f54a4fd92522f29f8b4d7b16f0b6

```
"""Fit-validation and retrospective errors for the two prediction methods."""
import json
import numpy as np,pandas as pd
from model import *

def paired(a,b,B=10000):
    aa=a.assign(se=(a.prediction-a.observed)**2).groupby(['event_id','event_year']).se.mean()
    bb=b.assign(se=(b.prediction-b.observed)**2).groupby(['event_id','event_year']).se.mean()
    z=pd.concat([aa.rename('a'),bb.rename('b')],axis=1).reset_index()
    years=np.sort(z.event_year.unique());codes=pd.Categorical(z.event_year,categories=years).codes
    rng=np.random.default_rng(20261024)
    counts=rng.multinomial(len(years),np.full(len(years),1/len(years)),size=B)[:,codes]
    diff=np.sqrt(counts@z.a/counts.sum(axis=1))-np.sqrt(counts@z.b/counts.sum(axis=1))
    return dict(RMSE_difference=float(np.sqrt(z.a.mean())-np.sqrt(z.b.mean())),
        lower=float(np.quantile(diff,.025)),upper=float(np.quantile(diff,.975)))

def main():
    d=pd.read_csv(R/'data/rebuilt_annual_states.csv');events=pd.read_csv(R/'data/events.csv').set_index('event_id')
    train=d[d.event_id.isin(events[events.used_for_fitting].index)]
    assert train.event_id.nunique()==36
    assert np.max(abs(fit_quadratic(train)-COEF))<1e-12
    folds={int(y):fit_quadratic(train[train.event_year.ne(y)]) for y in sorted(train.event_year.unique())}
    (R/'results/validation_coefficients.json').write_text(json.dumps({str(k):v.tolist() for k,v in folds.items()},indent=2))
    rows=[]
    for eid,g in d.groupby('event_id'):
        info=events.loc[eid];coef=folds[int(info.event_year)] if info.used_for_fitting else COEF
        for method in METHODS:
            q=event_predictions(g,coef,method);q['sample']=info.analysis_use;rows.append(q)
    allp=pd.concat(rows,ignore_index=True);allp.to_csv(R/'results/all_case_predictions.csv',index=False)
    test=allp[allp['sample'].eq('test')];assert test.event_id.nunique()==37
    test.to_csv(R/'results/test_predictions.csv',index=False)
    summaries=[];annual=[]
    for sample,frame in list(allp.groupby('sample'))+[('all_cases_descriptive',allp)]:
        for (method,target),g in frame.groupby(['method','target']):summaries.append(dict(sample=sample,method=method,target=target,**metrics(g)))
        for (method,target,t),g in frame[frame.t.ge(2)&frame.observed.notna()].groupby(['method','target','t']):annual.append(dict(sample=sample,method=method,target=target,t=int(t),**metrics(g)))
    pd.DataFrame(summaries).to_csv(R/'results/model_metrics.csv',index=False)
    pd.DataFrame(annual).to_csv(R/'results/annual_metrics.csv',index=False)
    cv=allp[allp['sample'].eq('training')&allp.method.eq('calibrated')&allp.t.ge(2)]
    widths=[]
    for (target,t),g in cv.groupby(['target','t']):
        err=np.sort(abs(g.prediction-g.observed).to_numpy());n=len(err)
        for coverage in [.8,.9,.95]:
            rank=int(np.ceil((n+1)*coverage))
            widths.append(dict(target=target,t=int(t),target_coverage=coverage,n=n,rank=rank,half_width=float(err[rank-1])))
    widths=pd.DataFrame(widths);widths.to_csv(R/'results/interval_half_widths.csv',index=False)
    coverage=[]
    for _,w in widths.iterrows():
        g=test[test.method.eq('calibrated')&test.target.eq(w.target)&test.t.eq(w.t)]
        coverage.append(dict(target=w.target,t=int(w.t),target_coverage=w.target_coverage,half_width=w.half_width,
            actual_coverage=float((abs(g.prediction-g.observed)<=w.half_width).mean()),n_fires=len(g)))
    pd.DataFrame(coverage).to_csv(R/'results/test_interval_coverage.csv',index=False)
    pairs=[]
    for target in TARGETS:
        a=test[test.method.eq('calibrated')&test.target.eq(target)&test.t.ge(2)]
        b=test[test.method.eq('quadratic')&test.target.eq(target)&test.t.ge(2)]
        pairs.append(dict(target=target,**paired(a,b)))
    pd.DataFrame(pairs).to_csv(R/'results/paired_comparisons.csv',index=False)
    grouped=test.merge(events[['trajectory_group']],left_on='event_id',right_index=True,validate='many_to_one');gm=[]
    for (group,method,target),g in grouped.groupby(['trajectory_group','method','target']):gm.append(dict(trajectory_group=group,method=method,target=target,**metrics(g)))
    pd.DataFrame(gm).to_csv(R/'results/trajectory_metrics.csv',index=False)
    complete=d.groupby('event_id').t.nunique();balanced=d[d.event_id.isin(complete[complete.eq(8)].index)]
    balanced.groupby('t')[list(TARGETS.values())].mean().reset_index().to_csv(R/'results/observed_means.csv',index=False)
    recent0=pd.read_csv(R/'data/recent_states.csv')
```

```
    recent=pd.concat([recent0[recent0.event_name.eq('PARK')],d[d.event_name.isin(['DIXIE','CALDOR','CREEK'])]],ignore_index=True)
    q=pd.concat([event_predictions(g,COEF,method) for _,g in recent.groupby('event_id') for method in METHODS],ignore_index=True)
    q.to_csv(R/'results/recent_predictions.csv',index=False)
    records=[]
    for (method,target),g in q[q.t.ge(2)&q.observed.notna()].groupby(['method','target']):records.append(dict(method=method,target=target,**metrics(g)))
    pd.DataFrame(records).to_csv(R/'results/recent_metrics.csv',index=False)
    print(pd.DataFrame(summaries).query("sample=='test'").pivot(index='method',columns='target',values='RMSE').mul(100).round(3))

if __name__=='__main__':main()
```

# 代码05 predict.py

SHA-256 103f7db29a1e292c306323fa9d11d0fd0305c97b06ba81ea98a714748e7e7d08

```
"""Predict years2-8 from two measured first-year relative states."""
import argparse,json
import numpy as np,pandas as pd
from model import R,predict

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--lg-first',type=float,required=True)
    parser.add_argument('--tcc-first',type=float,required=True)
    args=parser.parse_args();first=[args.lg_first,args.tcc_first]
    if not np.isfinite(first).all() or min(first)<0:parser.error('Inputs must be finite non-negative relative states.')
    years=np.arange(2,9);p,flags=predict(first,years)
    widths=pd.read_csv(R/'results/interval_half_widths.csv');rows=[]
    for j,age in enumerate(years):
        for k,target in enumerate(['LG','ZP_TCC','DE']):
            row=dict(t=int(age),target=target,prediction=float(p[j,k]),units='relative state')
            for coverage in [.8,.9,.95]:
                h=float(widths[widths.target.eq(target)&widths.t.eq(age)&widths.target_coverage.eq(coverage)].half_width.iloc[0])
                row[str(int(coverage*100))+'_empirical_interval']=[float(p[j,k]-h),float(p[j,k]+h)]
            rows.append(row)
    print(json.dumps(dict(**flags,forecasts=rows),ensure_ascii=False,indent=2))

if __name__=='__main__':main()
```

# 代码06 make_figures.py

SHA-256 5939b774eb3c4de46f00bacb88beddd9abdccdac3bdcb7a70d37773aef671193

```
"""Annual observations and prediction curves for forest fires."""
from pathlib import Path
import json,sys
import numpy as np,pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from PIL import Image
from matplotlib.font_manager import fontManager
from matplotlib.backends.backend_pdf import PdfPages
from matplotlib.patches import Patch

R=Path(__file__).resolve().parent
OUT=R.parent.parent/'output/forest_recovery_report'
from model import predict,quadratic,COEF
fontManager.addfont(str(R/'assets/NotoSansCJKsc-Regular.otf'))
plt.rcParams.update({'font.family':'Noto Sans CJK SC','font.size':9,'axes.spines.top':False,'axes.spines.right':False,'axes.unicode_minus':False})
BLUE='#286D98';GREY='#959FA5';BLACK='#252A2D'
TARGETS=[('LG','R_LG','LG相对状态'),('ZP_TCC','R_ZP_TCC','树冠覆盖相对状态'),('DE','DE','DE')]

def save_image(fig,path,**kwargs):
    temporary=path.with_suffix('.pending.png')
    fig.savefig(temporary,**kwargs)
    with Image.open(temporary) as check:check.load()
    temporary.replace(path)

def main():
    OUT.mkdir(parents=True,exist_ok=True);(R/'figures').mkdir(exist_ok=True)
    coef=COEF
    folds={int(k):np.array(v) for k,v in json.loads((R/'results/validation_coefficients.json').read_text()).items()}
    roles=pd.read_csv(R/'data/events.csv').set_index('event_id')
    d=pd.read_csv(R/'data/rebuilt_annual_states.csv')
    widths=pd.read_csv(R/'results/interval_half_widths.csv');plotted=[]
    complete=d.groupby('event_id').t.nunique();balanced=d[d.event_id.isin(complete[complete.eq(8)].index)]
    mean=balanced.groupby('t')[['R_LG','R_ZP_TCC','DE']].mean()
    fig,axs=plt.subplots(1,2,figsize=(10.3,3.7))
    for col,label,color in [('R_LG','LG',BLUE),('R_ZP_TCC','树冠覆盖','#B66F25')]:axs[0].plot(mean.index,100*mean[col],'o-',label=label,color=color)
    axs[0].set(xlabel='火后年份',ylabel='相对状态（点）',xticks=range(1,9));axs[0].legend(frameon=False)
    for _,g in balanced.groupby('event_id'):axs[1].plot(g.t,g.DE*100,color=GREY,alpha=.12,lw=.7)
    axs[1].plot(mean.index,mean.DE*100,'o-',color=BLUE,label='77场平均');axs[1].axhline(0,color=GREY,lw=.6,ls=':')
    axs[1].set(xlabel='火后年份',ylabel='DE（点）',xticks=range(1,9));axs[1].legend(frameon=False)
    for ax in axs:ax.grid(alpha=.12)
    fig.tight_layout();save_image(fig,R/'figures/mean_states.png',dpi=220,bbox_inches='tight');plt.close(fig)

    def draw(axs,g,number=None,recent=False):
        meta=g.iloc[0];eid=meta.event_id
        training=not recent and bool(roles.loc[eid,'used_for_fitting'])
        c=folds[int(meta.event_year)] if training else coef
        first=g[g.t.eq(1)].iloc[0];anchor=[[first.R_LG,first.R_ZP_TCC]]
        t=np.linspace(1,8,150);p,_=predict(anchor[0],t,coef=c);p=p[None,:,:2];base=quadratic(c,t)
        future=np.linspace(2,8,121);center,_=predict(anchor[0],future,coef=c);center=center[None,:,:2]
        for j,(ax,(k,col,label)) in enumerate(zip(axs,TARGETS)):
            y=p[0,:,j] if j<2 else p[0,:,1]-p[0,:,0]
            b=base[:,j] if j<2 else base[:,1]-base[:,0]
            cent=center[0,:,j] if j<2 else center[0,:,1]-center[0,:,0]
            for cov,alpha in [(.95,.07),(.9,.13),(.8,.26)]:
                w=widths[widths.target.eq(k)&widths.target_coverage.eq(cov)].sort_values('t')
                h=np.interp(future,w.t,w.half_width)
                ax.fill_between(future,(cent-h)*100,(cent+h)*100,color=BLUE,alpha=alpha,linewidth=0)
            ax.axvspan(.87,1.13,color='#EFF1F2',zorder=0)
            ax.plot(t,b*100,'--',color=GREY,lw=1.2,label='二次函数')
            ax.plot(t,y*100,color=BLUE,lw=1.5,label='幅度上限校正')
            ax.plot(g.t,g[col]*100,'o-',color=BLACK,lw=1,ms=2.5,label='遥感观测')
            ax.set(xlim=(.85,8.15),xticks=[1,2,4,6,8],xlabel='火后年份',ylabel=label+'（点）')
            ax.grid(alpha=.12);ax.tick_params(labelsize=8)
            if k=='DE':ax.axhline(0,color=GREY,lw=.6,ls=':')
        title=('' if number is None else f'{number:02d}  ')+str(meta.event_name)+' '+str(int(meta.
```

```
event_year))
        if recent:status='近期火灾应用'
        else:
            title+='  '+roles.loc[eid,'trajectory_group']
            if training:status='训练火灾；按起火年份交叉验证'
            elif roles.loc[eid,'included_in_test_score']:status='检验火灾；回顾性检验'
            else:status='补充观测火灾'
        axs[0].set_title(title+'\n'+status,loc='left',fontsize=7.4,pad=9)
        for ax in axs[1:]:ax.set_title(' ',pad=9)
        axs[0].legend(frameon=False,fontsize=6,loc='best')
        pp,_=predict(anchor[0],g.t.to_numpy(),coef=c);pp=pp[None,:,:2];bb=quadratic(c,g.t.to_numpy
())
        for j,(_,v) in enumerate(g.iterrows()):
            for k,(target,col,_) in enumerate(TARGETS):
                plotted.append(dict(event_id=eid,t=int(v.t),target=target,observed=float(v[col]),
                    prediction=float(pp[0,j,k] if k<2 else pp[0,j,1]-pp[0,j,0]),
                    quadratic=float(bb[j,k] if k<2 else bb[j,1]-bb[j,0]),recent=recent))
    inventory=roles.reset_index();inventory['order']=inventory.trajectory_group.map({'DE下降':0,'
基本不变':1,'DE上升':2,'观测未满8年':3})
    inventory=inventory.sort_values(['order','event_year','event_name','event_id']).reset_index(dr
op=True)
    inventory['case_number']=np.arange(1,len(inventory)+1)
    inventory.to_csv(R/'results/curve_inventory.csv',index=False)
    with PdfPages(OUT/'forest_v5_all_case_curves.pdf') as pdf:
        for start in range(0,len(inventory),3):
            fig,axs=plt.subplots(3,3,figsize=(10.3,11.1));sub=inventory.iloc[start:start+3]
            for j,(_,case) in enumerate(sub.iterrows()):draw(axs[j],d[d.event_id.eq(case.event_id)
].sort_values('t'),int(case.case_number))
            for j in range(len(sub),3):
                for ax in axs[j]:ax.axis('off')
            fig.subplots_adjust(hspace=.66,wspace=.34,top=.92,bottom=.105,left=.08,right=.98)
            fig.legend(handles=[Patch(facecolor=BLUE,alpha=a,label=f'目标{p}%范围') for p,a in [(8
0,.40),(90,.20),(95,.07)]],loc='upper center',ncol=3,frameon=False,fontsize=8)
            fig.text(.08,.060,f'案例 {start+1}–{min(start+3,len(inventory))}／81。LG与树冠覆盖分别
预测，DE为二者之差。',fontsize=7.4,color='#515B62')
            fig.text(.08,.040,'灰：二次函数；蓝：幅度上限校正；黑：实际观测。训练与回顾性检验结果
分别报告。',fontsize=7.5,color='#515B62')
            fig.text(.08,.021,'阴影半宽按本校正规则的训练留出误差计算，第2–8年所有案例同宽；第一年
不计分，未来观测不填补。',fontsize=7.5,color='#515B62')
            pdf.savefig(fig);plt.close(fig)
    recent=pd.read_csv(R/'data/recent_states.csv')
    recent=pd.concat([recent[recent.event_name.eq('PARK')],d[d.event_name.isin(['DIXIE','CALDOR','
CREEK'])]],ignore_index=True)
    for i,names in enumerate([['PARK','DIXIE'],['CALDOR','CREEK']],1):
        fig,axs=plt.subplots(2,3,figsize=(9.4,6.6))
        for axes,name in zip(axs,names):draw(axes,recent[recent.event_name.eq(name)].sort_values('
t'),recent=True)
        fig.subplots_adjust(hspace=.66,wspace=.35,top=.9,bottom=.1,left=.08,right=.99)
        save_image(fig,R/'figures'/f'recent_{i}.png',dpi=220,bbox_inches='tight');plt.close(fig)
    pd.DataFrame(plotted).drop_duplicates(['event_id','t','target']).to_csv(R/'results/plot_values
.csv',index=False)
    print('Rendered81 fire cases in27 atlas pages.')

if __name__=='__main__':main()
```

# 代码07 build_report.py

SHA-256 be4ede22758a3f290325105bdaeb00ae013aab9d64c5ee188e97b3e3cb927f49

```
"""Populate the research report from its layout and the numerical results."""
from pathlib import Path
from copy import deepcopy
from io import BytesIO
import pandas as pd
from PIL import Image
from docx import Document
from model import R,TARGETS,METHODS
OUT=R.parents[1]/'output/forest_recovery_report'

def set_text(p,s):
    props=deepcopy(p.runs[0]._r.rPr) if p.runs and p.runs[0]._r.rPr is not None else None
    for e in list(p._p):
        if not e.tag.endswith('}pPr'):p._p.remove(e)
    run=p.add_run(str(s))
    if props is not None:run._r.insert(0,props)

def fill(t,rows):
    while len(t.rows)<len(rows):t._tbl.append(deepcopy(t.rows[-1]._tr))
    while len(t.rows)>len(rows):t._tbl.remove(t.rows[-1]._tr)
    for row,values in zip(t.rows,rows):
        for c,v in zip(row.cells,values):
            set_text(c.paragraphs[0],v)
            for p in list(c.paragraphs)[1:]:p._p.getparent().remove(p._p)

def main():
    OUT.mkdir(parents=True,exist_ok=True)
    d=Document(R/'assets/report_layout.docx');m=pd.read_csv(R/'results/model_metrics.csv')
    a=pd.read_csv(R/'results/annual_metrics.csv');cov=pd.read_csv(R/'results/test_interval_coverage.csv')
    recent=pd.read_csv(R/'results/recent_predictions.csv');names={'quadratic':'二次函数','calibrated':'幅度上限校正'}
    for ix,sample in [(3,'training'),(4,'test')]:
        rows=[['方案','LG RMSE','树冠覆盖RMSE','DE RMSE']]
        for method in METHODS:
            g=m[m['sample'].eq(sample)&m.method.eq(method)].set_index('target')
            rows.append([names[method],*[f'{g.loc[k].RMSE*100:.2f}' for k in TARGETS]])
        fill(d.tables[ix],rows)
    rows=[['目标','方案','RMSE','MAE','偏差','R²']]
    for method in METHODS:
        for target,label in [('LG','LG'),('ZP_TCC','树冠覆盖'),('DE','DE')]:
            q=m[m['sample'].eq('test')&m.method.eq(method)&m.target.eq(target)].iloc[0]
            rows.append([label,names[method],*[f'{q[k]*100:.2f}' for k in ['RMSE','MAE','bias']],f'{q.R2:.3f}'])
    fill(d.tables[5],rows)
    for ix,target in [(6,'LG'),(7,'ZP_TCC'),(8,'DE')]:
        rows=[['火后年份','二次RMSE','上限校正RMSE','观测标准差','上限校正R²']]
        for age in range(2,9):
            g=a[a['sample'].eq('test')&a.target.eq(target)&a.t.eq(age)].set_index('method');v=g.loc['calibrated']
            rows.append([age,f'{g.loc["quadratic"].RMSE*100:.2f}',f'{v.RMSE*100:.2f}',f'{v.observed_SD*100:.2f}',f'{v.R2:.3f}'])
        fill(d.tables[ix],rows)
    rows=[['火灾 第八年日历年','LG预测','ZP预测','DE预测']]
    for name in ['PARK','DIXIE','CALDOR','CREEK']:
        g=recent[recent.event_name.eq(name)&recent.t.eq(8)&recent.method.eq('calibrated')].set_index('target')
        rows.append([f'{name} {int(g.calendar_year.iloc[0])}',*[f'{g.loc[k].prediction*100:.2f}' for k in TARGETS]])
    fill(d.tables[10],rows)
    for ix,target in [(13,'DE'),(14,'LG'),(15,'ZP_TCC')]:
        rows=[['火后年份','80%目标','90%目标','95%目标']]
        for age in range(2,9):
            g=cov[cov.target.eq(target)&cov.t.eq(age)].set_index('target_coverage')
            rows.append([age,*[f'±{g.loc[p].half_width*100:.2f}／{g.loc[p].actual_coverage*100:.1f}%' for p in [.8,.9,.95]]])
        fill(d.tables[ix],rows)
    pairs=pd.read_csv(R/'results/paired_comparisons.csv').set_index('target')
    for p in d.paragraphs:
        if p.text.startswith('按起火年份整组重抽样，'):
            z=pairs.loc['ZP_TCC'];de=pairs.loc['DE']
            set_text(p,f'按起火年份整组重抽样，校正相对二次函数的树冠覆盖RMSE差为{z.RMSE_difference*100:.2f}点，95%范围为{z.lower*100:.2f}至{z.upper*100:.2f}点；DE差为{de.RMSE_difference*100:.2f}点，范围为{de.lower*100:.2f}至{de.upper*100:.2f}点。这些范围表达本批样本的不确定性。')
    for rid,name in [('rId13','mean_states.png'),('rId15','recent_1.png'),('rId16','recent_2.png')]:d.part.related_parts[rid]._blob=(R/'figures'/name).read_bytes()
```

```
    for part in d.part.related_parts.values():
        if part.content_type.startswith('image/'):
            with Image.open(BytesIO(part.blob)) as image:image.load()
    d.save(OUT/'forest_v5_full_report(1).docx');print('Complete research report saved.')

if __name__=='__main__':main()
```

# 代码08 build_manual.py

SHA-256 0286e2851011e0f8e9b08b9b38a781cd4f0c9583a14030a995d5f463ed3f413b

```
"""Technical definitions, two prediction methods and their evaluation."""
import json
import pandas as pd
from model import *
OUT=R.parents[1]/'output/forest_recovery_report'

def table(headers,rows):
    return '\n'.join(['| '+' | '.join(map(str,headers))+' |','| '+' | '.join(['---']*len(headers))+' |']+['| '+' | '.join(map(str,r))+' |' for r in rows])

def main():
    m=pd.read_csv(R/'results/model_metrics.csv');a=pd.read_csv(R/'results/annual_metrics.csv');cov=pd.read_csv(R/'results/test_interval_coverage.csv')
    names={'quadratic':'二次函数','calibrated':'幅度上限校正'}
    text=['# 植被绿度与树冠覆盖 指标和模型说明','## 研究对象',
      '81场森林火灾，36场用于函数拟合和按起火年份交叉验证，37场用于回顾性检验，8场用于补充观测核对。77场具有完整八年观测。预测限火后1–8年，误差评价限第2–8年。',
      '## 指标定义',
      '物理ZP是单位面积内胸径至少10厘米活立木的断面积密度：ZP_phys=(10000/A)Σπ(DBH_i/200)²，单位平方米/公顷。它是地面可测量的结构指标，不参与时间函数拟合。',
      'LG=clip[-0.14910055653818355+0.5546205992959745NDVI+0.72735184813157O4EVI,0,1]，为DIRECT参照校准的绿色覆盖遥感估计。clip仅限制原始LG估计，不限制标准化相对状态。ZP_TCC=TCC/100，为USFS Science树冠覆盖比例，不等于活树数量、树高或活立木断面积。',
      'I_k(t)、C_k(t)是固定受灾和对照位置的年度平均。q_k=I_k/C_k，b_k=[q_k(-3)+q_k(-2)+q_k(-1)]/3，R_k=q_k/b_k。DE=R_ZP_TCC-R_LG。展示值乘100，单位为点；相对状态可以大于1。',
      'NDVI=(ρ_NIR-ρ_R)/(ρ_NIR+ρ_R)；EVI=2.5(ρ_NIR-ρ_R)/(ρ_NIR+6ρ_R-7.5ρ_B+1)。ρ为各波段反射率，NIR、R、B分别为近红外、红光和蓝光。MOD13Q1指数整数乘0.0001；每年6–8月SummaryQA≤1的记录分别取中位数。',
      'TCC为30米年度Science产品，版本2025-6；有效data_mask=1的面积平均到250米网格，再平均固定位置。每年使用相同位置。缺测不由预测值补齐。',
      '## 二次函数',
      'f_k(t)=a_k+b_kt+c_kt²。两项分别拟合，DE预测由两项相减。',
      table(['指标','a','b','c'],[[k,*[f'{v:.17g}' for v in CONFIG['coefficients'][k]]] for k in ['LG','ZP_TCC']]),
      '## 幅度上限校正',
      'e_k=R_k(1)-f_k(1)，Rhat_k(t)=f_k(t)+s_k(t)。首年ZP不在[0.46,0.64]时s_ZP=e_ZP，否则s_ZP=0.25e_ZP。',
      '只有ZP在上述区间外且abs(e_LG)≥0.12时，s_LG=sign(e_LG)×min(abs(e_LG),abs(s_ZP))；其他情形s_LG(t)=e_LG×0.5^(t-1)。sign保留LG偏差的正负方向，min取绝对值较小者。LG校正不必通过首年实际观测点。',
      'DEhat=Rhat_ZP_TCC-Rhat_LG，没有第三套差值回归。第1年是输入，不计入预测误差。']
    for sample,title in [('training','训练交叉验证'),('test','37场回顾性检验')]:
        rows=[]
        for method in METHODS:
            for target in TARGETS:
                q=m[m['sample'].eq(sample)&m.method.eq(method)&m.target.eq(target)].iloc[0]
                rows.append([names[method],target,int(q.n_fires),int(q.n_states),*[f'{q[k]*100:.3f}' for k in ['RMSE','MAE','bias','observed_SD']],f'{q.R2:.4f}'])
        text += ['## '+title,table(['方案','指标','火灾数','年度数','RMSE','MAE','偏差','SD','R²'],rows)]
    text+=['训练验证按起火年份整体留出。研究为回顾性分析，资料也参与了校正规则的确定；回顾性检验不等于严格前瞻盲测，不能保证新事件具有同样精度。',
      '## 每个火后年份的结果']
    for target in TARGETS:
        rows=[]
        for age in range(2,9):
            q=a[a['sample'].eq('test')&a.target.eq(target)&a.t.eq(age)].set_index('method');v=q.loc['calibrated']
            rows.append([age,*[f'{x*100:.3f}' for x in [q.loc['quadratic'].RMSE,v.RMSE,v.MAE,v.bias,v.observed_SD]],f'{v.R2:.4f}'])
        text += ['### '+target,table(['火后年','二次RMSE','校正RMSE','校正MAE','校正偏差','观测SD','校正R²'],rows)]
    text += ['## 经验预测范围','各指标、各年份用36个训练交叉验证绝对误差排序，取ceil((36+1)×p)个误差为半宽。p=80%、90%、95%分别取第30、34、36个值。检验数据仅用于计算实际覆盖率。相同指标、年份和目标对所有案例使用同一半宽。']
    for target in TARGETS:
        rows=[]
        for age in range(2,9):
            q=cov[cov.target.eq(target)&cov.t.eq(age)].set_index('target_coverage')
            rows.append([age,*[f'±{q.loc[p].half_width*100:.3f}／{q.loc[p].actual_coverage*100:.1f}%' for p in [.8,.9,.95]]])
        text += ['### '+target,table(['火后年','80%半宽／覆盖','90%半宽／覆盖','95%半宽／覆盖'],rows)]
    text += ['标注概率是经验目标。95%目标使用36个误差中的最大值，小样本可能产生宽区间。区间不限制在0–100点，不代表完整生态恢复的可能比例。',
      '## 误差计算','先在各火灾的已有第2–8年计算平均平方误差、绝对误差及有符号误差，再在火灾间等权。RMSE为均方误差的平方根，偏差为预测减观测。SD使用相同权重的总体描述方差，不作N−1修正，R²=1-RMSE²/
```

```
    SD²。负R²保留。',
0052        '## 复现','代码位于analysis/forest_recovery，依次运行rebuild_measurements.py、analyze.py、ma
    ke_figures.py、build_report.py、build_manual.py、verify.py和build_code_evidence.py。输入、输出与运
    行说明见README.md。',
0053        '## 作者贡献与AI工具使用','作者负责研究问题、指标与模型形式、首年校正及幅度上限思路、正式方
    法选择，以及解释和结论审定。ChatGPT协助搜集公开资料、处理核对数据，编写获取数据、统计、作图和复现
    程序，并整理研究资料。最终内容与结论由作者核查并负责。']
0054      (OUT/'model_and_index_manual.md').write_text('\n\n'.join(text)+'\n');print('Model and index ma
    nual saved.')
0055
0056  if __name__=='__main__':main()
```

# 代码09 verify.py

SHA-256 a549f8eda788dd7df0369645b5691a635c36fa77c9982d088e7f08c1fd94d2fb

```
"""Check measurements, the two methods, numerical tables and report images."""
from pathlib import Path
from io import BytesIO
import hashlib,json
import numpy as np,pandas as pd
from PIL import Image
from docx import Document
from model import *
OUT=R.parents[1]/'output/forest_recovery_report'

def main():
    tests=[]
    def check(condition,label):
        assert condition,label
        tests.append(label)
    d=pd.read_csv(R/'data/rebuilt_annual_states.csv');events=pd.read_csv(R/'data/events.csv')
    p=pd.read_csv(R/'results/all_case_predictions.csv');m=pd.read_csv(R/'results/model_metrics.csv')
    a=pd.read_csv(R/'results/annual_metrics.csv');w=pd.read_csv(R/'results/interval_half_widths.csv')
    check(len(events)==81 and len(d)==636,'81 fires and636 annual measurements')
    check(events.used_for_fitting.sum()==36 and events.included_in_test_score.sum()==37,'36 training and37 test fires')
    check(set(p.method)==set(METHODS),'Two prediction methods')
    check(p.groupby(['method','target']).event_id.nunique().eq(81).all(),'81 fires forecast by both methods')
    check(p.groupby(['method','target','event_id']).t.nunique().eq(8).all(),'Eight forecast years for each fire')
    check(abs(d.DE-(d.R_ZP_TCC-d.R_LG)).max()<1e-12,'Observed DE component difference')
    for (sample,method,target),g in p[p.t.ge(2)&p.observed.notna()].groupby(['sample','method','target']):
        independent=np.sqrt(g.assign(se=(g.prediction-g.observed)**2).groupby('event_id').se.mean().mean())
        row=m[m['sample'].eq(sample)&m.method.eq(method)&m.target.eq(target)].iloc[0]
        check(abs(independent-row.RMSE)<1e-12,f'{sample}/{method}/{target}: independent RMSE')
        check(abs(row.R2-(1-row.RMSE**2/row.observed_SD**2))<1e-12,f'{sample}/{method}/{target}: R2')
    test=p[p['sample'].eq('test')&p.t.ge(2)]
    check(test.groupby(['method','target']).size().eq(259).all(),'37x7 test observations per target and method')
    for _,row in w.iterrows():
        g=p[p['sample'].eq('training')&p.method.eq('calibrated')&p.target.eq(row.target)&p.t.eq(row.t)]
        err=np.sort(abs(g.prediction-g.observed));rank=int(np.ceil((len(g)+1)*row.target_coverage))
        check(len(g)==36 and abs(row.half_width-err[rank-1])<1e-12,f'Training error interval {row.target}/{row.t}/{row.target_coverage}')
    pv=pd.read_csv(R/'results/plot_values.csv')
    joined=pv.merge(p[p.method.eq('calibrated')][['event_id','t','target','prediction']],on=['event_id','t','target'],suffixes=('','_table'))
    check(joined.event_id.nunique()==81 and abs(joined.prediction-joined.prediction_table).max()<1e-12,'Plot values and prediction table agree')
    for (method,eid),g in p.groupby(['method','event_id']):
        q=g.pivot(index='t',columns='target',values='prediction')
        check(abs(q.DE-(q.ZP_TCC-q.LG)).max()<1e-12,f'{method}/{eid}: forecast DE from components')
    doc=Document(OUT/'forest_v5_full_report(1).docx')
    paragraphs=[q.text for q in doc.paragraphs]
    contribution=next(i for i,x in enumerate(paragraphs) if x=='作者贡献与AI工具使用说明')
    bibliography=next(i for i,x in enumerate(paragraphs) if x.startswith('9 参考文献'))
    check(contribution<bibliography,'Author contributions before bibliography')
    check(any(x.startswith('作者负责') for x in paragraphs),'Human contribution statement')
    check(any(x.startswith('ChatGPT协助') for x in paragraphs),'AI technical assistance statement')
    for rid,part in doc.part.related_parts.items():
        if part.content_type.startswith('image/'):
            with Image.open(BytesIO(part.blob)) as image:image.load()
            check(True,'Complete image '+rid)
    for ix in [3,4]:check(len(doc.tables[ix].rows)==3,f'Table{ix}: exactly two methods')
    for path,h in json.loads((R/'data_sha256.json').read_text()).items():
        check(hashlib.sha256((R/path).read_bytes()).hexdigest()==h,'Data checksum '+path)
    result=dict(status='PASS',checks=len(tests),details=tests,n_fires=81,n_training=36,n_test=37,methods=METHODS)
    (R/'results/verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2))
    print(json.dumps({k:v for k,v in result.items() if k!='details'},ensure_ascii=False,indent=2))

```

```
0060  if __name__=='__main__':main()
```

# 代码10 collect_satellite.js

SHA-256 65da82195fedcb7ac007581a8d9ef9aa0ea57e68c287ffb317a2e907daa36795

```
// Import data/fixed_positions.geojson as an Earth Engine table named fixed_positions.
// The table is an input; account and project identifiers are configured in GEE.
var VI=ee.ImageCollection('MODIS/061/MOD13Q1');
var TCC=ee.ImageCollection('projects/gtac-data-publish/assets/TCC/Product_Version/2025-6')
  .filter(ee.Filter.eq('study_area','CONUS'));
var projection=ee.ImageCollection('MODIS/061/MOD44B').first()
  .select('Percent_Tree_Cover').projection();
var targetProj=projection.atScale(250);
function blank(names){
  return ee.Image.constant(ee.List.repeat(0,names.length)).rename(names)
    .updateMask(ee.Image(0)).setDefaultProjection(targetProj);
}
function annual(year){
  var source=VI.filter(ee.Filter.calendarRange(year,year,'year'))
    .filter(ee.Filter.calendarRange(6,8,'month'));
  var good=source.map(function(im){return im.select(['NDVI','EVI']).multiply(.0001)
    .updateMask(im.select('SummaryQA').lte(1));});
  var strict=source.map(function(im){return im.select(['NDVI','EVI']).multiply(.0001)
    .updateMask(im.select('SummaryQA').eq(0));});
  var green=good.median().rename(['ndvi','evi'])
    .addBands(good.count().rename(['ndvi_n','evi_n']))
    .addBands(strict.count().rename(['ndvi_n_q0','evi_n_q0']));
  var tc=TCC.filter(ee.Filter.eq('year',year));
  var image=ee.Image(ee.Algorithms.If(tc.size().gt(0),tc.first(),
    blank(['Science_Percent_Tree_Canopy_Cover',
      'Science_Percent_Tree_Canopy_Cover_Standard_Error','data_mask'])));
  var valid=image.select('data_mask').eq(1);
  var fine=image.select(['Science_Percent_Tree_Canopy_Cover',
    'Science_Percent_Tree_Canopy_Cover_Standard_Error'],['tcc30','tcc30_product_se'])
    .updateMask(valid);
  var canopy=fine.reduceResolution({reducer:ee.Reducer.mean(),maxPixels:4096})
    .reproject(targetProj);
  var fraction=valid.unmask(0).reduceResolution({reducer:ee.Reducer.mean(),maxPixels:4096})
    .reproject(targetProj).rename('tcc30_mapped_fraction');
  return green.addBands(canopy).addBands(fraction).addBands(ee.Image.pixelLonLat()).unmask(-9999);
}
function exportMeasurements(points,startYear,endYear){
  var panel=ee.FeatureCollection(ee.List.sequence(startYear,endYear).map(function(year){
    year=ee.Number(year);
    var active=points.filter(ee.Filter.gte('event_year',year.subtract(8)))
      .filter(ee.Filter.lte('event_year',year.add(3)));
    return annual(year).sampleRegions({collection:active,
      properties:['event_id','event_name','event_year','group','pixel_id','x','y','severity','pref
ire_lc'],
      projection:projection,scale:250,tileScale:8,geometries:false})
      .map(function(f){return f.set({year:year,t:year.subtract(ee.Number(f.get('event_year'))),
        tcc_version:'2025-6'});});
  })).flatten();
  Export.table.toDrive({collection:panel,description:'forest_fire_measurements',fileFormat:'CSV'})
;
}
exportMeasurements(fixed_positions,2001,2026);
```

# 代码11 collect_green_reference.js

SHA-256 93064dd28e72ef008abae8d2731f3f81cb85dec9d2b8835286f91d8693f57d68

```
0001  // Independent DIRECT normal-forest reference matching, 3 km support.
0002  var refs=ee.FeatureCollection({"type":"FeatureCollection","features":[{"type":"Feature","geometry"
      :{"type":"Polygon","coordinates":[[[116.23824484276776,-32.61187009303069],[116.27020725723227,-32
      .61187009303069],[116.27020245569352,-32.5848180455603],[116.2382496443065,-32.5848180455603],[116
      .23824484276776,-32.61187009303069]]]},"properties":{"ref_id":0,"siteid":52,"site":"Camerons ","la
      t":-32.5983451,"lon":116.25422605,"date":"2004-03-03","reference":0.414,"landcover":"EBF"}},{"type
      ":"Feature","geometry":{"type":"Polygon","coordinates":[[[-53.25132577169236,5.333578028657068],[-
      53.22425990830764,5.333578028657068],[-53.22425931253744,5.3607067467884],[-53.25132636746256,5.36
      07067467884],[-53.25132577169236,5.333578028657068]]]},"properties":{"ref_id":1,"siteid":54,"site"
      :"Counami ","lat":5.34714254,"lon":-53.23779284,"date":"2001-09-26","reference":0.86,"landcover":"
      EBF"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[-53.25036285107883,5.329896
      487138816],[-53.22329714892116,5.329896487138816],[-53.22329655356708,5.357025208517436],[-53.2503
      6344643292,5.357025208517436],[-53.25036285107883,5.329896487138816]]]},"properties":{"ref_id":2,"
      siteid":55,"site":"Counami2 ","lat":5.343461,"lon":-53.23683,"date":"2002-10-13","reference":0.86,
      "landcover":"EBF"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[115.8665699412
      22,-31.547377309142703],[115.89816405877804,-31.547377309142703],[115.89815950430523,-31.520320711
      745715],[115.8665744956948,-31.520320711745715],[115.866569941222,-31.547377309142703]]]},"propert
      ies":{"ref_id":3,"siteid":59,"site":"Gnangara ","lat":-31.53385,"lon":115.882367,"date":"2004-03-0
      1","reference":0.221,"landcover":"EBF"}},{"type":"Feature","geometry":{"type":"Polygon","coordinat
      es":[[[26.98237715828123,62.630538709356706],[27.04084284171877,62.630538709356706],[27.0408693595
      8397,62.65745516320707],[26.982350640416033,62.65745516320707],[26.98237715828123,62.6305387093567
      06]]]},"properties":{"ref_id":4,"siteid":62,"site":"Hirsikangas ","lat":62.644,"lon":27.01161,"dat
      e":"2003-08-14","reference":0.644,"landcover":"NLF"}},{"type":"Feature","geometry":{"type":"Polygo
      n","coordinates":[[[26.98237715828123,62.630538709356706],[27.04084284171877,62.630538709356706],[
      27.04086935958397,62.65745516320707],[26.982350640416033,62.65745516320707],[26.98237715828123,62.
      630538709356706]]]},"properties":{"ref_id":5,"siteid":62,"site":"Hirsikangas ","lat":62.644,"lon":
      27.01161,"date":"2004-07-08","reference":0.537,"landcover":"NLF"}},{"type":"Feature","geometry":{"
      type":"Polygon","coordinates":[[[26.98237715828123,62.630538709356706],[27.04084284171877,62.63053
      8709356706],[27.04086935958397,62.65745516320707],[26.982350640416033,62.65745516320707],[26.98237
      715828123,62.630538709356706]]]},"properties":{"ref_id":6,"siteid":62,"site":"Hirsikangas ","lat":
      62.644,"lon":27.01161,"date":"2005-06-08","reference":0.64,"landcover":"NLF"}},{"type":"Feature","
      geometry":{"type":"Polygon","coordinates":[[[27.236731062216833,58.28519337097326],[27.28787151778
      317,58.28519337097326],[27.287890950492205,58.31212748510218],[27.236711629507795,58.3121274851021
      8],[27.236731062216833,58.28519337097326]]]},"properties":{"ref_id":7,"siteid":64,"site":"Jarvselj
      a ","lat":58.298663,"lon":27.26230129,"date":"2000-07-06","reference":0.754,"landcover":"Mixed F."
      }},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[27.236731062216833,58.285193370
      97326],[27.28787151778317,58.28519337097326],[27.287890950492205,58.31212748510218],[27.2367116295
      07795,58.31212748510218],[27.236731062216833,58.28519337097326]]]},"properties":{"ref_id":8,"sitei
      d":64,"site":"Jarvselja ","lat":58.298663,"lon":27.26230129,"date":"2001-06-14","reference":0.783,
      "landcover":"Mixed F."}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[27.236731
      062216833,58.28519337097326],[27.28787151778317,58.28519337097326],[27.287890950492205,58.31212748
      510218],[27.236711629507795,58.31212748510218],[27.236731062216833,58.28519337097326]]]},"properti
      es":{"ref_id":9,"siteid":64,"site":"Jarvselja ","lat":58.298663,"lon":27.26230129,"date":"2002-06-
      27","reference":0.793,"landcover":"Mixed F."}},{"type":"Feature","geometry":{"type":"Polygon","coo
      rdinates":[[[27.236731062216833,58.28519337097326],[27.28787151778317,58.28519337097326],[27.28789
      0950492205,58.31212748510218],[27.236711629507795,58.31212748510218],[27.236731062216833,58.285193
      37097326]]]},"properties":{"ref_id":10,"siteid":64,"site":"Jarvselja ","lat":58.298663,"lon":27.26
      230129,"date":"2003-07-27","reference":0.803,"landcover":"Mixed F."}},{"type":"Feature","geometry"
      :{"type":"Polygon","coordinates":[[[27.236731062216833,58.28519337097326],[27.28787151778317,58.28
      519337097326],[27.287890950492205,58.31212748510218],[27.236711629507795,58.31212748510218],[27.23
      6731062216833,58.28519337097326]]]},"properties":{"ref_id":11,"siteid":64,"site":"Jarvselja ","lat
      ":58.298663,"lon":27.26230129,"date":"2005-06-29","reference":0.842,"landcover":"Mixed F."}},{"typ
      e":"Feature","geometry":{"type":"Polygon","coordinates":[[[27.236731062216833,58.28519337097326],[
      27.28787151778317,58.28519337097326],[27.287890950492205,58.31212748510218],[27.236711629507795,58
      .31212748510218],[27.236731062216833,58.28519337097326]]]},"properties":{"ref_id":12,"siteid":64,"
      site":"Jarvselja ","lat":58.298663,"lon":27.26230129,"date":"2007-04-22","reference":0.53,"landcov
      er":"Mixed F."}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[27.23673106221683
      3,58.28519337097326],[27.28787151778317,58.28519337097326],[27.287890950492205,58.31212748510218],
      [27.236711629507795,58.31212748510218],[27.236731062216833,58.28519337097326]]]},"properties":{"re
      f_id":13,"siteid":64,"site":"Jarvselja ","lat":58.298663,"lon":27.26230129,"date":"2007-07-18","re
      ference":0.7,"landcover":"Mixed F."}},{"type":"Feature","geometry":{"type":"Polygon","coordinates"
      :[[[-75.23614787523964,45.367071792672526],[-75.19785412476037,45.367071792672526],[-75.1978450117
      8046,45.39406496420196],[-75.23615698821953,45.39406496420196],[-75.23614787523964,45.367071792672
      526]]]},"properties":{"ref_id":14,"siteid":66,"site":"Larose ","lat":45.38057,"lon":-75.217001,"da
      te":"2003-08-07","reference":0.847,"landcover":"Mixed F."}},{"type":"Feature","geometry":{"type":"
      Polygon","coordinates":[[[-1.056368499551951,44.554479908639635],[-1.0186115004480494,44.554479908
      639635],[-1.018602766135873,44.5814769374401],[-1.0563772338641273,44.5814769374401],[-1.056368499
      551951,44.554479908639635]]]},"properties":{"ref_id":15,"siteid":68,"site":"Nezer ","lat":44.56798
      ,"lon":-1.03749,"date":"2000-07-29","reference":0.541,"landcover":"NLF"}},{"type":"Feature","geome
      try":{"type":"Polygon","coordinates":[[[-1.056368499551951,44.554479908639635],[-1.018611500448049
      4,44.554479908639635],[-1.018602766135873,44.5814769374401],[-1.0563772338641273,44.5814769374401]
      ,[-1.056368499551951,44.554479908639635]]]},"properties":{"ref_id":16,"siteid":68,"site":"Nezer ",
      "lat":44.56798,"lon":-1.03749,"date":"2001-04-09","reference":0.415,"landcover":"NLF"}},{"type":"F
      eature","geometry":{"type":"Polygon","coordinates":[[[-1.056368499551951,44.554479908639635],[-1.0
      186115004480494,44.554479908639635],[-1.018602766135873,44.5814769374401],[-1.0563772338641273,44.
      5814769374401],[-1.056368499551951,44.554479908639635]]]},"properties":{"ref_id":17,"siteid":68,"s
```

```
ite":"Nezer ","lat":44.56798,"lon":-1.03749,"date":"2001-06-24","reference":0.871,"landcover":"NLF
"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[-1.056368499551951,44.55447990
8639635],[-1.0186115004480494,44.554479908639635],[-1.018602766135873,44.5814769374401],[-1.056377
2338641273,44.5814769374401],[-1.056368499551951,44.554479908639635]]]},"properties":{"ref_id":18,
"siteid":68,"site":"Nezer ","lat":44.56798,"lon":-1.03749,"date":"2002-04-17","reference":0.45,"la
ndcover":"NLF"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[3.633291750159205
7,43.7110802726984],[3.670515189840794,43.7110802726984],[3.670523551153398,43.738081303415846],[3
.6332833888466025,43.738081303415846],[3.6332917501592057,43.7110802726984]]]},"properties":{"ref_
id":19,"siteid":70,"site":"Puechabon ","lat":43.72458232,"lon":3.65190347,"date":"2001-06-13","ref
erence":0.54,"landcover":"Mixed F."}},{"type":"Feature","geometry":{"type":"Polygon","coordinates"
:[[[25.31741566019196,66.44219515044529],[25.38465433980804,66.44219515044529],[25.38469054781499,
66.46909758661599],[25.317379452185012,66.46909758661599],[25.31741566019196,66.44219515044529]]]}
,"properties":{"ref_id":20,"siteid":72,"site":"Rovaniemi ","lat":66.45565,"lon":25.351035,"date":"
2004-06-09","reference":0.423,"landcover":"NLF"}},{"type":"Feature","geometry":{"type":"Polygon","
coordinates":[[[25.31741566019196,66.44219515044529],[25.38465433980804,66.44219515044529],[25.384
69054781499,66.46909758661599],[25.317379452185012,66.46909758661599],[25.31741566019196,66.442195
15044529]]]},"properties":{"ref_id":21,"siteid":72,"site":"Rovaniemi ","lat":66.45565,"lon":25.351
035,"date":"2005-06-15","reference":0.497,"landcover":"NLF"}},{"type":"Feature","geometry":{"type"
:"Polygon","coordinates":[[[4.389824562718518,50.75466493358059],[4.432337217281482,50.75466493358
059],[4.432349441104892,50.78163271911185],[4.3898123388951085,50.78163271911185],[4.3898245627185
18,50.75466493358059]]]},"properties":{"ref_id":22,"siteid":73,"site":"Sonian ","lat":50.76815078,
"lon":4.41108089,"date":"2004-06-22","reference":0.903,"landcover":"NLF"}},{"type":"Feature","geom
etry":{"type":"Polygon","coordinates":[[[24.27913274527164,61.83786498021226],[24.33608415472836,6
1.83786498021226],[24.33610913349216,61.864784532392456],[24.279107766507835,61.864784532392456],[
24.27913274527164,61.83786498021226]]]},"properties":{"ref_id":23,"siteid":81,"site":"Hyyti\u00e4l
\u00e4","lat":61.85132772,"lon":24.30760845,"date":"2008-07-06","reference":0.614,"landcover":"NLF
"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[25.481060129210228,-18.6621621
79888668],[25.509495426345374,-18.662162179888668],[25.509493170431,-18.63505894761447],[25.481062
385124606,-18.63505894761447],[25.481060129210228,-18.662162179888668]]]},"properties":{"ref_id":2
4,"siteid":82,"site":"Pandamatenga","lat":-18.648611111111112,"lon":25.495277777777778,"date":"2000
-03-04","reference":0.39,"landcover":"DBF"}},{"type":"Feature","geometry":{"type":"Polygon","coord
inates":[[[13.571941313250884,41.836493612596385],[13.608058686749118,41.836493612596385],[13.6080
66284887302,41.8635035150508],[13.5719337151127,41.8635035150508],[13.571941313250884,41.836493612
596385]]]},"properties":{"ref_id":25,"siteid":126,"site":"Collelongo","lat":41.85,"lon":13.59,"dat
e":"2015-07-08","reference":0.84,"landcover":"DBF"}},{"type":"Feature","geometry":{"type":"Polygon
","coordinates":[[[13.571941313250884,41.836493612596385],[13.608058686749118,41.836493612596385],
[13.6080066284887302,41.8635035150508],[13.5719337151127,41.8635035150508],[13.571941313250884,41.8
36493612596385]]]},"properties":{"ref_id":26,"siteid":126,"site":"Collelongo","lat":41.85,"lon":13
.59,"date":"2015-09-25","reference":0.86,"landcover":"DBF"}},{"type":"Feature","geometry":{"type":
"Polygon","coordinates":[[[-0.7180140720180145,39.73839878693328],[-0.6830159279819856,39.73839878
693328],[-0.6830090900687452,39.76541854249272],[-0.7180209099312549,39.76541854249272],[-0.718014
0720180145,39.73839878693328]]]},"properties":{"ref_id":27,"siteid":130,"site":"Liria","lat":39.75
191,"lon":-0.700515,"date":"2017-02-09","reference":0.271,"landcover":"NLF"}},{"type":"Feature","g
eometry":{"type":"Polygon","coordinates":[[[-1.345511316533248,51.751691202940165],[-1.30206916346
6752,51.751691202940165],[-1.3020562195899172,51.778654370230335],[-1.3455242604100828,51.77865437
0230335],[-1.345511316533248,51.751691202940165]]]},"properties":{"ref_id":28,"siteid":132,"site":
"Wyhtam","lat":51.76517481,"lon":-1.32379024,"date":"2018-07-06","reference":0.6743,"landcover":"D
BF"}},{"type":"Feature","geometry":{"type":"Polygon","coordinates":[[[-1.345511316533248,51.751691
202940165],[-1.302069163466752,51.751691202940165],[-1.3020562195899172,51.778654370230335],[-1.34
55242604100828,51.778654370230335],[-1.345511316533248,51.751691202940165]]]},"properties":{"ref_i
d":29,"siteid":132,"site":"Wyhtam","lat":51.76517481,"lon":-1.32379024,"date":"2021-07-18","refere
nce":0.8447,"landcover":"DBF"}}]};
0003  var vi=ee.ImageCollection('MODIS/061/MOD13Q1');var lai=ee.ImageCollection('MODIS/061/MOD15A2H');
0004  function summarize(im,geom,names){var a=ee.Image.pixelArea();var v=im.multiply(a).addBands(a.updat
      eMask(im.select(0).mask()).rename('valid_area'));
0005   var s=v.reduceRegion({reducer:ee.Reducer.sum(),geometry:geom,crs:im.select(0).projection(),maxPix
      els:1e7,tileScale:4});
0006   var ar=ee.Number(ee.Algorithms.If(s.get('valid_area'),s.get('valid_area'),0));var out=ee.Dictiona
      ry({valid_fraction:ar.divide(geom.area(1))});
0007   names.forEach(function(n){out=out.set(n,ee.Algorithms.If(ar.gt(0),ee.Number(s.get(n)).divide(ar),
      null));});return out;}
0008  var rows=refs.map(function(f){var date=ee.Date(f.get('date'));var y=ee.Number(date.get('year'));va
      r doy=date.getRelative('day','year').add(1);
0009   var vs=ee.Date.fromYMD(y,1,1).advance(doy.subtract(1).divide(16).floor().multiply(16),'day');var
      ls=ee.Date.fromYMD(y,1,1).advance(doy.subtract(1).divide(8).floor().multiply(8),'day');
0010   var vv=vi.filterDate(vs,vs.advance(1,'day'));var vb=ee.Image.constant([0,0,0,0,0]).rename(['NDVI'
      ,'EVI','SummaryQA','DetailedQA','DayOfYear']).updateMask(ee.Image(0)).setDefaultProjection(ee.Imag
      e(vi.first()).select(0).projection());
0011   var v=ee.Image(ee.Algorithms.If(vv.size().gt(0),vv.first(),vb));var day=v.select('DayOfYear');var
       dd=day.subtract(doy).abs();dd=dd.min(ee.Image(366).subtract(dd));
0012   var vg=v.select(['NDVI','EVI']).multiply(.0001).updateMask(v.select('SummaryQA').eq(0).and(v.sele
      ct('DetailedQA').bitwiseAnd(3).eq(0)).and(dd.lte(8)));
0013   var ll=lai.filterDate(ls,ls.advance(1,'day'));var lb=ee.Image.constant([0,0,0,0]).rename(['Lai_50
      0m','Fpar_500m','FparLai_QC','FparExtra_QC']).updateMask(ee.Image(0)).setDefaultProjection(ee.Imag
      e(lai.first()).select(0).projection());
0014   var l=ee.Image(ee.Algorithms.If(ll.size().gt(0),ll.first(),lb));var q=l.select('FparLai_QC'),ex=l
      .select('FparExtra_QC');
0015   var good=q.bitwiseAnd(1).eq(0).and(q.rightShift(5).bitwiseAnd(7).lte(1)).and(q.rightShift(3).bitw
      iseAnd(3).eq(0)).and(ex.rightShift(2).bitwiseAnd(1).eq(0));
0016   var lg=l.select('Lai_500m').multiply(.1).rename('LAI').addBands(l.select('Fpar_500m').multiply(.0
      1).rename('FPAR')).updateMask(good);
0017   var a=summarize(vg,f.geometry(),['NDVI','EVI']);var b=summarize(lg,f.geometry(),['LAI','FPAR']);
```

```
0018   return ee.Feature(null,f.toDictionary()).set(a).set('VI_valid_fraction',a.get('valid_fraction')).
      set(b).set('LAI_valid_fraction',b.get('valid_fraction')).set({VI_start:vs.format('YYYY-MM-dd'),LAI
      _start:ls.format('YYYY-MM-dd')});});
0019  print('Independent forest references',refs.size());
0020  Export.table.toDrive({collection:rows,description:'forest_green_reference_v2',folder:'Forest_recov
      ery_research_20261022',fileNamePrefix:'forest_green_reference',fileFormat:'CSV'});
0021  rows.getDownloadURL('csv',null,'forest_green_reference',function(u,e){print(e||ui.Label('DOWNLOAD_
      DIRECT_MATCHED',{},u));});
```

# 代码12 build_code_evidence.py

SHA-256 198cf159acb6fe701968880eaf99cd33d059c18ca4eaed906bfdfe79fc0c694e

```
"""Full executable source and actual data excerpts for the standalone study."""
from pathlib import Path
import hashlib,html,json,re,unicodedata
import numpy as np,pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.font_manager import fontManager
from reportlab.platypus import SimpleDocTemplate,Paragraph,Spacer,PageBreak,Image,Table,TableStyle,XPreformatted
from reportlab.lib.styles import getSampleStyleSheet,ParagraphStyle
from reportlab.lib import colors
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.cidfonts import UnicodeCIDFont
from reportlab.pdfbase.ttfonts import TTFont
from model import R,CONFIG,TARGETS,METHODS
ROOT=R.parents[1];OUT=ROOT/'output/forest_recovery_report';FIG=R/'evidence'
CODE_FILES=['model.py','rebuild_measurements.py','check_green_calibration.py','analyze.py',
    'predict.py','make_figures.py','build_report.py','build_manual.py','verify.py',
    'collect_satellite.js','collect_green_reference.js','build_code_evidence.py','package_release.py']
fontManager.addfont(str(R/'assets/NotoSansCJKsc-Regular.otf'))
plt.rcParams.update({'font.family':'Noto Sans CJK SC','font.size':9,'axes.unicode_minus':False})
pdfmetrics.registerFont(UnicodeCIDFont('STSong-Light'))
pdfmetrics.registerFont(TTFont('CodeMono','/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf'))

def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()

def table_image(frame,path,title,subtitle):
    rows=frame.fillna('缺测').copy()
    for col in rows:rows[col]=rows[col].map(lambda x:f'{x:.4f}' if isinstance(x,(float,np.floating)) else str(x))
    fig,ax=plt.subplots(figsize=(11.3,max(2.6,.36*len(rows)+1.2)));ax.axis('off')
    ax.set_title(title,loc='left',fontsize=13,pad=28)
    tb=ax.table(cellText=rows.values,colLabels=rows.columns,loc='center',cellLoc='center')
    tb.auto_set_font_size(False);tb.set_fontsize(8.5);tb.scale(1,1.55)
    for (i,j),cell in tb.get_celld().items():
        cell.set_edgecolor('#D5DCE0');cell.set_linewidth(.5)
        cell.set_facecolor('#EAF0F4' if i==0 else 'white')
    fig.text(.02,.015,subtitle,fontsize=8,color='#4A5359')
    fig.savefig(path,dpi=180,bbox_inches='tight');plt.close(fig)

def wrap(line,width=98):
    parts=[];chunk='';used=0
    for char in line.expandtabs(4):
        advance=2 if unicodedata.east_asian_width(char) in ['W','F'] else 1
        if used+advance>width:parts.append(chunk);chunk='';used=0
        chunk+=char;used+=advance
    return parts+[chunk]

def code_markup(text):
    return re.sub(r'([\u3000-\u303f\u3400-\u9fff\uff00-\uffef]+)',
        lambda m:'<font name="STSong-Light">'+m.group(1)+'</font>',html.escape(text))

def main():
    FIG.mkdir(exist_ok=True);OUT.mkdir(exist_ok=True)
    pix=pd.read_csv(R/'data/satellite_pixel_years.csv',low_memory=False)
    groups=pd.read_csv(R/'data/group_year_values.csv');states=pd.read_csv(R/'data/rebuilt_annual_states.csv')
    events=pd.read_csv(R/'data/events.csv');metrics=pd.read_csv(R/'results/model_metrics.csv')
    willow=pix[pix.event_name.eq('WILLOW')&pix.group.eq('impact')];pid=willow.pixel_id.iloc[0]
    sample=willow[willow.pixel_id.eq(pid)&willow.t.isin([-3,-2,-1,1,2,8])][
        ['year','t','ndvi','evi','ndvi_n','tcc30','tcc30_mapped_fraction']].sort_values('t')
    table_image(sample,FIG/'01_satellite_values.png','WILLOW固定受灾位置的卫星记录',
        '实际CSV摘录；位置ID：'+str(pid)+'。TCC为百分比。')
    audits=[]
    for name in ['DAY','WILLOW','WHISKEY COMPLEX']:
        match=events[events.event_name.eq(name)]
        if match.empty:continue
        eid=match.event_id.iloc[0];g=groups[groups.event_id.eq(eid)];s=states[states.event_id.eq(eid)].set_index('t')
        for age in [1,2,8]:
            q=g[g.t.eq(age)].set_index('group');v=s.loc[age]
            audits.append(dict(火灾=name,火后年=age,受灾LG=q.loc['impact','LG'],对照LG=q.loc['control','LG'],
                受灾TCC=q.loc['impact','ZP_TCC']*100,对照TCC=q.loc['control','ZP_TCC']*100,
```

```
                LG状态=v.R_LG*100,ZP状态=v.R_ZP_TCC*100,DE=v.DE*100))
    table_image(pd.DataFrame(audits),FIG/'02_group_values.png','典型火灾的组均值与相对状态',
        'LG原始值为比例，TCC为百分比，状态与DE为点。完整记录保存在CSV中。')
    baseline=pd.read_csv(R/'data/baseline_values.csv')
    view=baseline[baseline.event_name.isin(['DAY','WILLOW','WHISKEY COMPLEX'])].drop(columns='event_id')
    table_image(view,FIG/'03_baselines.png','灾前三年基准核对','CV为三年样本标准差／均值绝对值。')
    recent=pd.read_csv(R/'data/recent_group_values.csv')
    view=recent[recent.event_name.eq('PARK')&recent.t.isin([-3,-2,-1,1,2])][
        ['group','year','t','LG','ZP_TCC','n_fixed','n_LG','n_TCC']].sort_values(['t','group'])
    table_image(view,FIG/'04_recent_measurements.png','PARK的观测与真实缺测',
        '2026年LG已有观测，同一产品版本的TCC缺测。观测空值未用预测填充。')
    view=metrics[metrics['sample'].eq('test')][['method','target','n_fires','n_states','RMSE','observed_SD','R2']].copy()
    view[['RMSE','observed_SD']]*=100
    table_image(view,FIG/'05_test_metrics.png','37场回顾性检验的评分输出',
        'quadratic为二次函数；calibrated为幅度上限校正。第2–8年，每目标259个状态。')
    sources=[R/'data/satellite_pixel_years.csv',R/'data/group_year_values.csv',R/'data/rebuilt_annual_states.csv',
        R/'data/baseline_values.csv',R/'data/recent_group_values.csv',R/'results/model_metrics.csv',
        R/'data/measurement/product_reextraction_checks.csv']
    code={str((R/n).relative_to(ROOT)):sha(R/n) for n in CODE_FILES}
    record=dict(code_SHA256=code,evidence_data_SHA256={str(p.relative_to(ROOT)):sha(p) for p in sources},
        figures_are_actual_CSV_excerpts=True,figures_are_browser_screenshots=False)
    (R/'results/code_evidence_manifest.json').write_text(json.dumps(record,ensure_ascii=False,indent=2))
    styles=getSampleStyleSheet()
    styles.add(ParagraphStyle(name='CN',fontName='STSong-Light',fontSize=10.5,leading=16,spaceAfter=9,wordWrap='CJK'))
    styles.add(ParagraphStyle(name='CNTitle',parent=styles['CN'],fontSize=20,leading=27,spaceAfter=18))
    styles.add(ParagraphStyle(name='CNH',parent=styles['CN'],fontSize=14,leading=20,spaceBefore=10,spaceAfter=10))
    styles.add(ParagraphStyle(name='Small',parent=styles['CN'],fontSize=8,leading=11))
    styles.add(ParagraphStyle(name='Source',fontName='CodeMono',fontSize=7,leading=9))
    story=[]
    def para(s,style='CN'):story.append(Paragraph(html.escape(str(s)),styles[style]))
    def heading(s):para(s,'CNH')
    def image_page(title,name,caption):
        story.append(PageBreak());heading(title)
        from PIL import Image as PILImage
        with PILImage.open(FIG/name) as im:w,h=im.size
        scale=min(490/w,510/h)
        story.append(Image(str(FIG/name),width=w*scale,height=h*scale));story.append(Spacer(1,12));para(caption)
    para('森林火灾研究完整代码与复现证据','CNTitle')
    para('森林火灾后植被绿度与树冠覆盖的变化差异及数学预测')
    para('本文件列出指标计算、二次函数、幅度上限校正、评分、作图与文件生成的完整程序，并提供实际数据摘录。全部可执行UTF-8源码和原始表随资料包提供。')
    heading('一 复现入口')
    para('保持analysis/forest_recovery与output/forest_recovery_report的相对目录。运行顺序及依赖见README.md；指标和单位见data_dictionary.md。数值复算使用保存的实际观测，不需要登录GEE；重新采集卫星资料需要使用自己的账户和固定位置文件。')
    para('输入包含103,764条位置年度记录、9,564个固定位置、81场火灾及636个火灾年度。36场用于拟合和按起火年份交叉验证，37场用于回顾性检验，8场用于补充核对。第一年作为校正输入，误差只评价第2–8年。')
    para('所有数据图证均为实际CSV记录的可视化摘录，不是GEE网页截图，也不是新增的地面观测。产品重提取核查表为保存的实际记录，运行本包不表示重新访问卫星平台。')
    heading('二 指标和正式函数')
    para('LG为DIRECT参照校准的绿色覆盖估计。ZP_TCC=TCC/100，为树冠覆盖代理。两者的受灾／对照年度比值分别除以灾前三年基准；DE=R_ZP_TCC−R_LG。物理ZP可以通过地面逐树调查测量，但不参与时间预测。')
    for k in ['LG','ZP_TCC']:para(k+'二次函数[a,b,c]：'+json.dumps(CONFIG['coefficients'][k]),'Small')
    para('首年偏差e_k=R_k(1)−f_k(1)，校正后Rhat_k(t)=f_k(t)+s_k(t)。ZP首年在[0.46,0.64]外，s_ZP=e_ZP；区间内s_ZP=0.25e_ZP。只有区间外且abs(e_LG)≥0.12时，s_LG=sign(e_LG)×min(abs(e_LG),abs(s_ZP))；其他情形s_LG=e_LG×0.5^(t−1)。DE预测由两项相减。')
    para('37场回顾性检验RMSE：二次函数LG12.661、ZP13.651、DE9.592点；幅度上限校正8.561、8.421、8.596点。训练按起火年份交叉验证RMSE分别为10.668、11.290、8.710点，以及9.261、7.082、8.877点。训练DE没有改善；结果不能视为普遍预测优势。')
    para('校正规则的确定使用了研究资料，检验属于回顾性分析。经验半宽只用训练交叉验证绝对误差计算，检验资料用于报告实际覆盖率。首年误差不计入改善，真实缺测不替换为预测。')
    for title,name,caption in [
        ('三 固定位置记录','01_satellite_values.png','源：data/satellite_pixel_years.csv。位置ID、质量次数和覆盖率均可逐行核对。'),
        ('四 组均值与年度状态','02_group_values.png','源：data/group_year_values.csv及rebuilt_annual_states.csv。先算受灾／对照比值，再除以三年基准。'),
        ('五 灾前基准','03_baselines.png','源：data/baseline_values.csv。不同指标分别使用自己的基准，不共用分母。'),
        ('六 近期火灾记录','04_recent_measurements.png','源：data/recent_group_values.csv。与数据库37场检验分别核对。'),
        ('七 相同样本评分','05_test_metrics.png','源：results/model_metrics.csv。RMSE与SD采用相同
```

```
    火灾等权；R²=1−RMSE²/SD²。')]:image_page(title,name,caption)
    story.append(PageBreak());heading('八 81场火灾的用途')
    labels={'training':'函数拟合与交叉验证','test':'回顾性检验','supplement':'补充观测核对'}
    rows=[['火灾','起火年','用途']]
    for _,v in events.sort_values(['analysis_use','event_year','event_name']).iterrows():
        rows.append([Paragraph(html.escape(v.event_name),styles['Small']),str(int(v.event_year)),P
    aragraph(labels[v.analysis_use],styles['Small'])])
    tb=Table(rows,colWidths=[295,55,145],repeatRows=1,hAlign='LEFT')
    tb.setStyle(TableStyle([('FONTNAME',(0,0),(-1,-1),'STSong-Light'),('FONTSIZE',(0,0),(-1,-1),8)
    ,
        ('BACKGROUND',(0,0),(-1,0),colors.HexColor('#EAF0F4')),('GRID',(0,0),(-1,-1),.4,colors.Hex
    Color('#D9D9D9')),
        ('VALIGN',(0,0),(-1,-1),'MIDDLE'),('TOPPADDING',(0,0),(-1,-1),4),('BOTTOMPADDING',(0,0),(-
    1,-1),4)]));story.append(tb)
    story.append(PageBreak());heading('九 代码与数据校验')
    para('SHA-256核对的是文件字节内容，不证明生态测量无误。全包校验值见MANIFEST_SHA256.json。')
    for name,h in record['evidence_data_SHA256'].items():para(name,'Small');para(h,'Small')
    heading('十 完整代码')
    para('下面所有程序未省略函数或代码行。PDF按阅读宽度自动折行；运行与复制应使用资料包同名源文件
    。固定参数见model.json；干净的独立报告版式见assets/report_layout.docx。')
    for i,name in enumerate(CODE_FILES,1):para(f'{i:02d} {name}','Small')
    for i,name in enumerate(CODE_FILES,1):
        story.append(PageBreak());heading(f'代码{i:02d} {name}')
        para('SHA-256 '+code[str((R/name).relative_to(ROOT))],'Small')
        lines=(R/name).read_text().splitlines()
        for start in range(0,len(lines),24):
            rendered=[]
            for j,line in enumerate(lines[start:start+24],start+1):
                chunks=wrap(line);rendered += [f'{j:04d}  '+chunks[0]]+['      '+s for s in chunks
    [1:]]
            story.append(XPreformatted(code_markup('\n'.join(rendered)),styles['Source']))
    story.append(PageBreak());heading('作者贡献与AI工具使用说明')
    para('作者负责研究问题与目标、指标和模型形式、首年校正及幅度上限思路、正式方法选择，以及解释和
    结论审定。')
    para('ChatGPT协助搜集公开数据和文献、处理与核对数据，以及编写数据获取、统计计算、作图和复现程
    序，并整理研究资料。论文最终内容与结论由作者核查并负责。')
    def footer(c,doc):
        c.setFont('STSong-Light',8);c.setFillColor(colors.HexColor('#4A5359'))
        c.drawString(45,25,'森林火灾研究 完整代码与复现证据');c.drawRightString(550,25,str(doc.pag
    e))
    SimpleDocTemplate(str(OUT/'forest_v5_code_and_evidence.pdf'),pagesize=(595.28,841.89),
        rightMargin=45,leftMargin=45,topMargin=45,bottomMargin=42,
        title='森林火灾研究完整代码与复现证据',author='研究复现资料').build(story,onFirstPage=foot
    er,onLaterPages=footer)
    print('Full code and evidence PDF saved.')

if __name__=='__main__':main()
```

# 代码13 package_release.py

SHA-256 17f6bd25b80b36b32711ab67371a979f928fc1a2415ecd9aee1e98e72212ba42

```
"""Package the standalone report, its two prediction methods and source data."""
from pathlib import Path
import datetime,hashlib,json,zipfile
R=Path(__file__).resolve().parent;ROOT=R.parents[1]
OUT=ROOT/'output/forest_recovery_report'
DELIVERABLES=['forest_v5_full_report(1).docx','forest_v5_full_report.pdf',
    'forest_v5_all_case_curves.pdf','model_and_index_manual.md','forest_v5_code_and_evidence.pdf']

def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()

def main():
    science=json.loads((R/'results/verification.json').read_text())
    visual=json.loads((OUT/'visual_verification.json').read_text())
    assert science['status']==visual['status']=='PASS'
    for name,h in visual['final_file_SHA256'].items():assert sha(OUT/name)==h,name
    evidence=json.loads((R/'results/code_evidence_manifest.json').read_text())
    for name,h in evidence['code_SHA256'].items():assert sha(ROOT/name)==h,name
    paths=[]
    for p in sorted(R.rglob('*')):
        if not p.is_file() or any(x.startswith('.') or x=='__pycache__' for x in p.relative_to(R).parts):continue
        if p.suffix in ['.pyc','.log']:continue
        paths.append(p)
    paths += [OUT/n for n in DELIVERABLES]+[OUT/'visual_verification.json']
    target=OUT/'forest_v5_reproducible_research.zip';manifest={}
    with zipfile.ZipFile(target,'w',zipfile.ZIP_DEFLATED,compresslevel=6) as z:
        for p in paths:
            name=p.relative_to(ROOT).as_posix();assert name not in manifest
            z.write(p,name);manifest[name]=sha(p)
        z.write(R/'README.md','README.md');manifest['README.md']=sha(R/'README.md')
        provenance=dict(created_UTC=datetime.datetime.now(datetime.timezone.utc).isoformat(),
            title='森林火灾后植被绿度与树冠覆盖的变化差异及数学预测',
            methods=['quadratic','calibrated'],time_window=[1,8],scoring_window=[2,8],
            source_fires=81,fitting_fires=36,test_fires=37,supplement_fires=8,
            test_type='retrospective',observations_imputed=False,
            scientific_verification_status=science['status'])
        blob=json.dumps(provenance,ensure_ascii=False,indent=2).encode()
        z.writestr('PROVENANCE.json',blob);manifest['PROVENANCE.json']=hashlib.sha256(blob).hexdigest()
        z.writestr('MANIFEST_SHA256.json',json.dumps(manifest,ensure_ascii=False,indent=2))
    with zipfile.ZipFile(target) as z:
        for name,h in manifest.items():assert hashlib.sha256(z.read(name)).hexdigest()==h,name
    result=dict(status='PASS',verified_files=len(manifest),archive_bytes=target.stat().st_size,archive_SHA256=sha(target))
    (OUT/'archive_verification.json').write_text(json.dumps(result,indent=2))
    print(json.dumps(result,indent=2))

if __name__=='__main__':main()
```

## 作者贡献与AI工具使用说明

作者负责研究问题与目标、指标和模型形式、首年校正及幅度上限思路、正式方法选择，以及解释和结论审定。

ChatGPT协助搜集公开数据和文献、处理与核对数据，以及编写数据获取、统计计算、作图和复现程序，并整理研究资料。论文最终内容与结论由作者核查并负责。